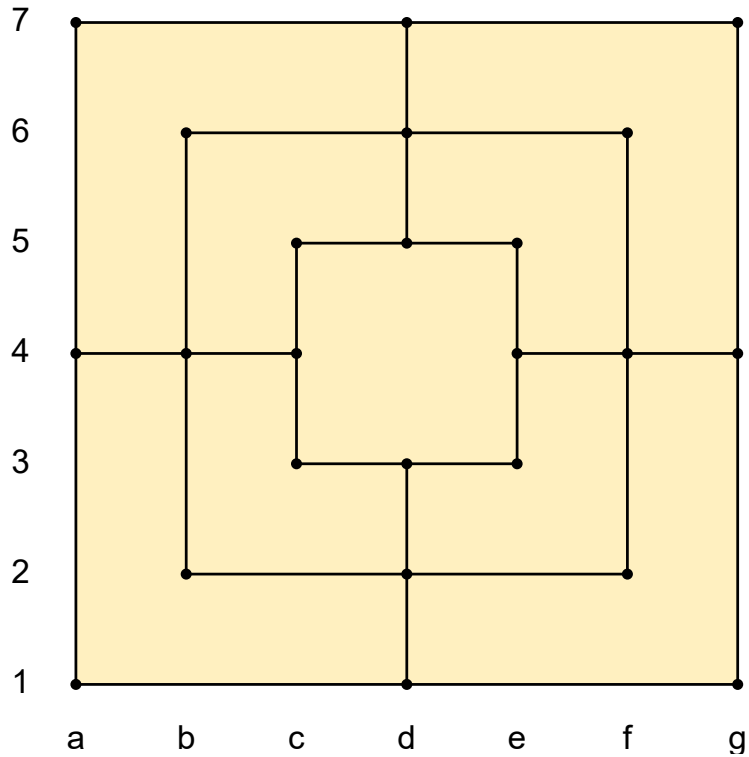
Młynek

Zasady gry i sposób reprezentacji pozycji

Jacek Klimaszewski

Plansza

- Gra toczy się na planszy złożonej z 24 pól.
- Na początku każdy gracz dysponuje dziewięcioma pionkami.
- Celem gry jest zdobycie siedmiu pionków przeciwnika lub uniemożliwienie mu ruchu.



Etapy gry

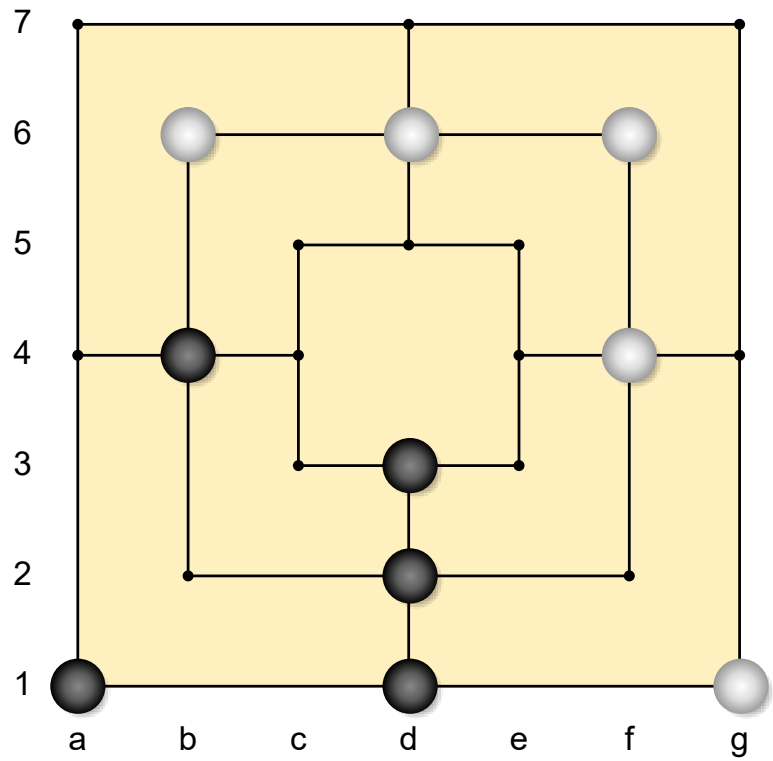
Gra składa się z dwóch etapów:

I Gracze naprzemiennie rozmieszczają swoje pionki na planszy. Jeżeli któryś gracz zbuduje „młynek” (układ trzech pionków w jednej linii), to zabiera przeciwnikowi pionka, który nie wchodzi w skład młynka (jeżeli wszystkie pionki przeciwnika tworzą młynki, to wtedy może zabrać dowolny z nich).

II Po wyłożeniu pionków gracze naprzemiennie wykonują ruchy swoimi pionkami. Pionki mogą przemieszczać się po liniach do sąsiednich niezajętych pól[1]. Ponownie można budować młynki.

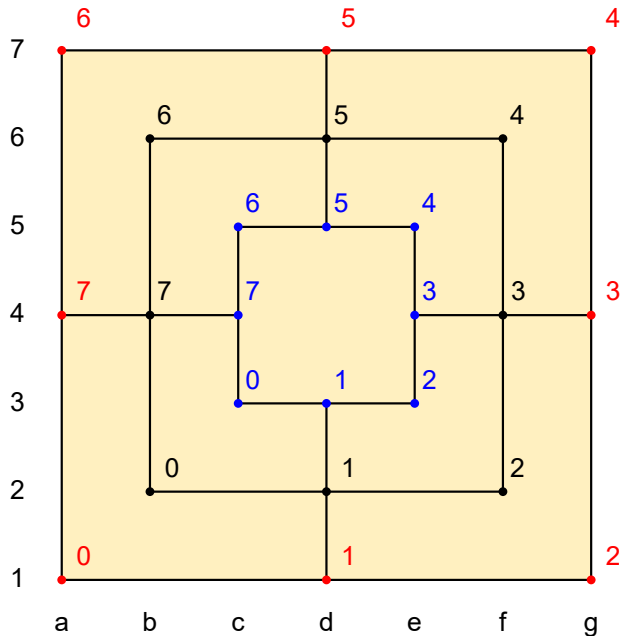
Przykład

Rysunek: Białe w pierwszej fazie gry umieściły pionka na polu `b6`, tworząc młynek, i dzięki temu mogą zabrać czarnego pionka z pola `a1` lub `b4` — czarne pionki tworzące młynek są nietykalne.



Tablica dwuwymiarowa

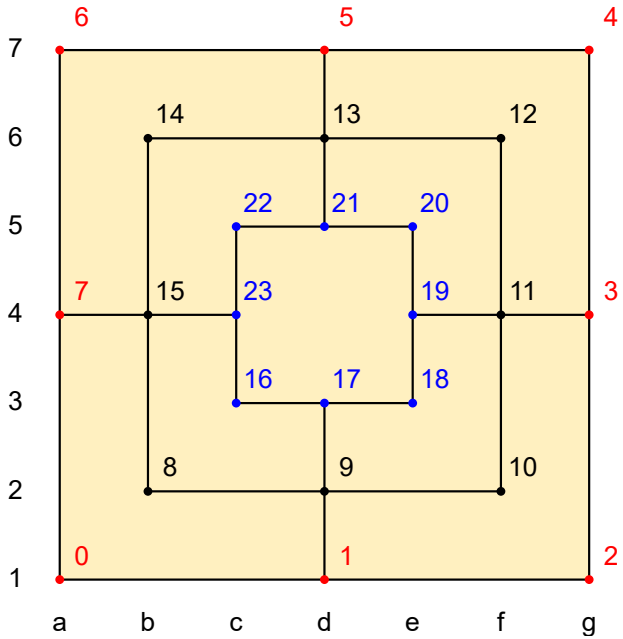
Plansza składa się z trzech kwadratów po osiem pól, zatem można ją reprezentować jako tablicę dwuwymiarową `[3][8]`, gdzie pierwszy indeks to numer kwadratu, a drugi to indeks pola. Na polu o indeksie parzystym można iść tylko do przodu ($j + 1 \mod 8$) lub do tyłu ($j - 1 \mod 8$), gdzie j to indeks pola (drugi indeks tablicy), natomiast na polu o indeksie nieparzystym można poruszać się też między kwadratami.



Ze względu na swoją prostotę jest to **zalecany** sposób reprezentacji — nie ustępuje znacząco dwom pozostałym możliwościom (tablicy jednowymiarowej i bitmapie) pod względem wydajności.

Tablica jednowymiarowa

Idąc krok dalej, tablicę dwuwymiarową można zastąpić tablicą jednowymiarową. Tutaj pomocna w generowaniu stanów potomnych może okazać się lista sąsiedztwa każdego pola, np. dla pola `17` mamy trzech sąsiadów {`9,16,18`}, dla pola `0` mamy dwóch sąsiadów {`1,7`} itd. Alternatywnie można zauważyć, że pola w środkowym kwadracie mają indeks o 8, a w wewnętrznym o 16 większy od indeksów pól w zewnętrznym kwadracie.



Lista figur

Zamiast trzymać całą planszę, można przechowywać jedynie położenie figur obu przeciwników (np. w postaci dwóch kontenerów `std::unordered_set`), jednakże stracimy na wydajności (sprawdzenie, czy pole jest puste, wymaga zajrzenia do obu kontenerów).

W przypadku skrajnym położenie figur można zakodować na pojedynczych bitach, czyli dwie liczby całkowite o długości trzydziestu dwu bitów wystarczyłyby do przechowywania położenia figur obu przeciwników (tzw. reprezentacja bitmapowa). Jest to najbardziej złożony sposób reprezentacji, ale też najwydajniejszy (używając odpowiednich masek można błyskawicznie sprawdzić za pomocą bitowej koniunkcji, czy pionek tworzy młynek).

Ruch jako napis

Ruch można zakodować za pomocą napisu złożonego z:

- jednej liczby — w ten sposób wskażemy pole, na które chcemy położyć pionka (na pierwszym etapie gry),
- dwóch liczb — w ten sposób wskażemy pole, na który chcemy położyć pionka i pole, z którego chcemy zabrać pionka przeciwnika (gdy utworzyliśmy młynek na pierwszym etapie gry) **lub** wskażemy pole, z którego chcemy ruszyć i na które chcemy przemieścić naszego pionka (na drugim etapie gry),
- trzech liczb — w ten sposób wskażemy pole, z którego chcemy ruszyć i na które chcemy przemieścić naszego pionka oraz pole, z którego chcemy zabrać pionka przeciwnika (gdy utworzyliśmy młynek).

Ruch zapisywany na bajtach

Ruch można zakodować na jednym, dwu lub trzech bajtach i przechowywać w liczbie o rozmiarze trzydziestu dwu bitów. Jeśli ktoś nie chce manipulować bajtami, to może skorzystać z unii:

```
union ruch
{
    uint32_t move;
    uint8_t m[4];
};
```

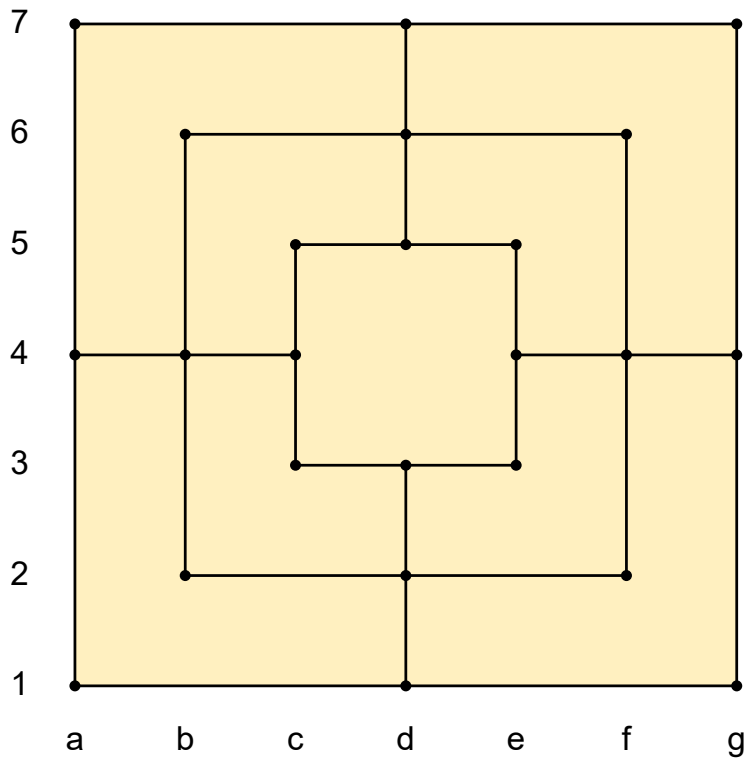
Pusty ruch możesz zapisać jako `~0u` w polu `move` (ustawione wszystkie bity — ma to na celu wyłapanie końca ciągu liczb), a następnie ustawić pierwszy, drugi bądź trzeci bajt, wpisując odpowiednią wartość do tablicy `m`, w zależności od rodzaju ruchu.

Przykład I

pierwszy etap gry

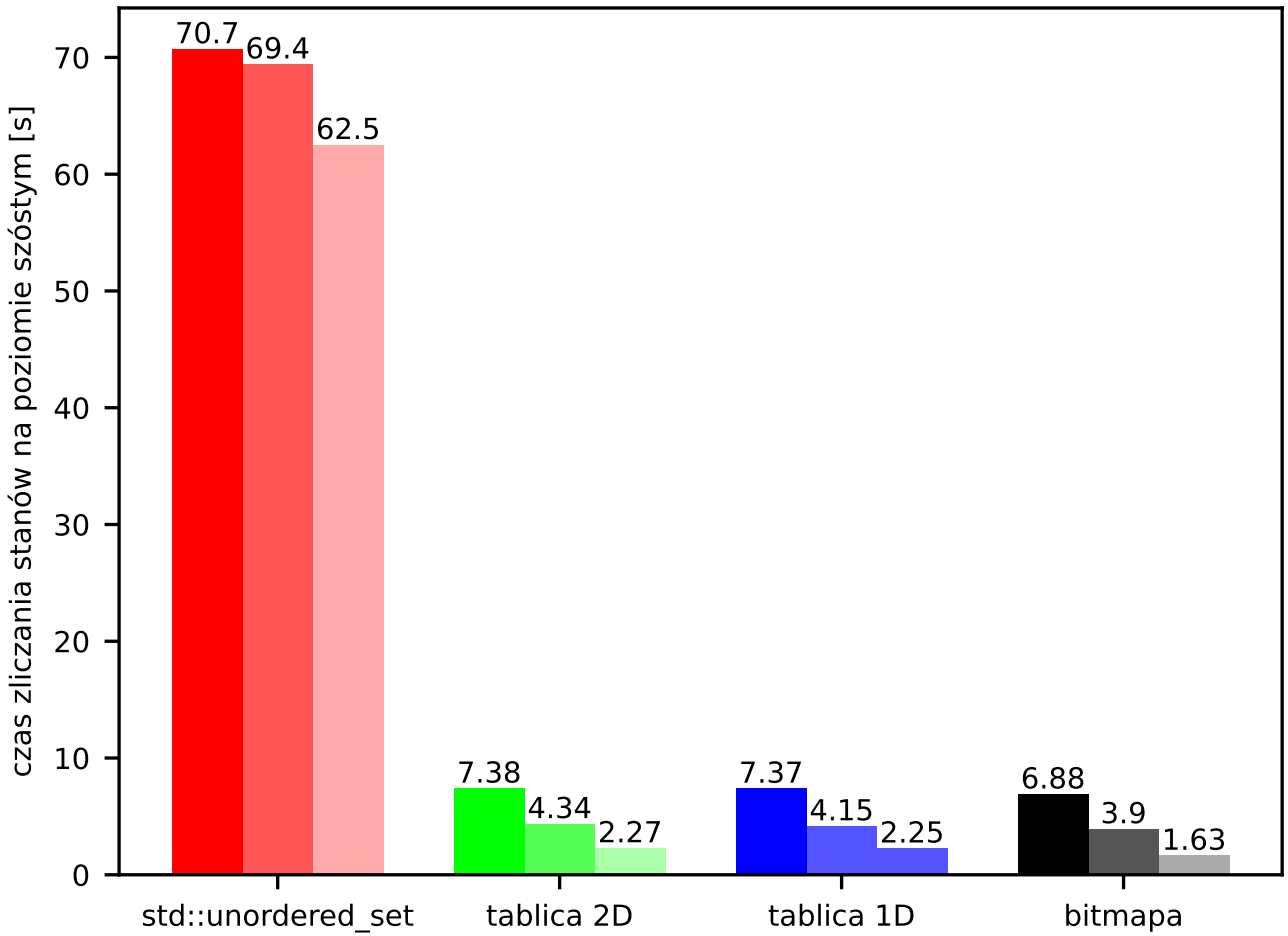
Liczba stanów na poszczególnych poziomach drzewa, zaczynając od podanej planszy (ruch białych):

głębokość	liczba stanów
1	24
2	552
3	12144
4	255024
5	5140800
6	99274176



Przykład I

porównanie wydajności poszczególnych sposobów reprezentacji



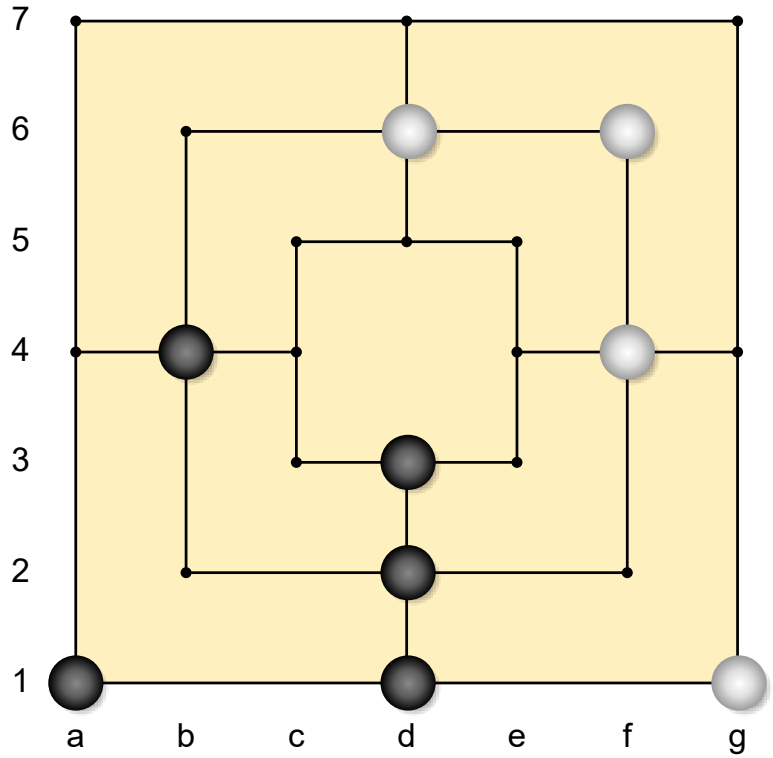
pierwszy słupek: `set_move(std::format(...))`; drugi słupek: `m_move += ...`; trzeci słupek: współrzędne w bajtach

Przykład II

pierwszy etap gry

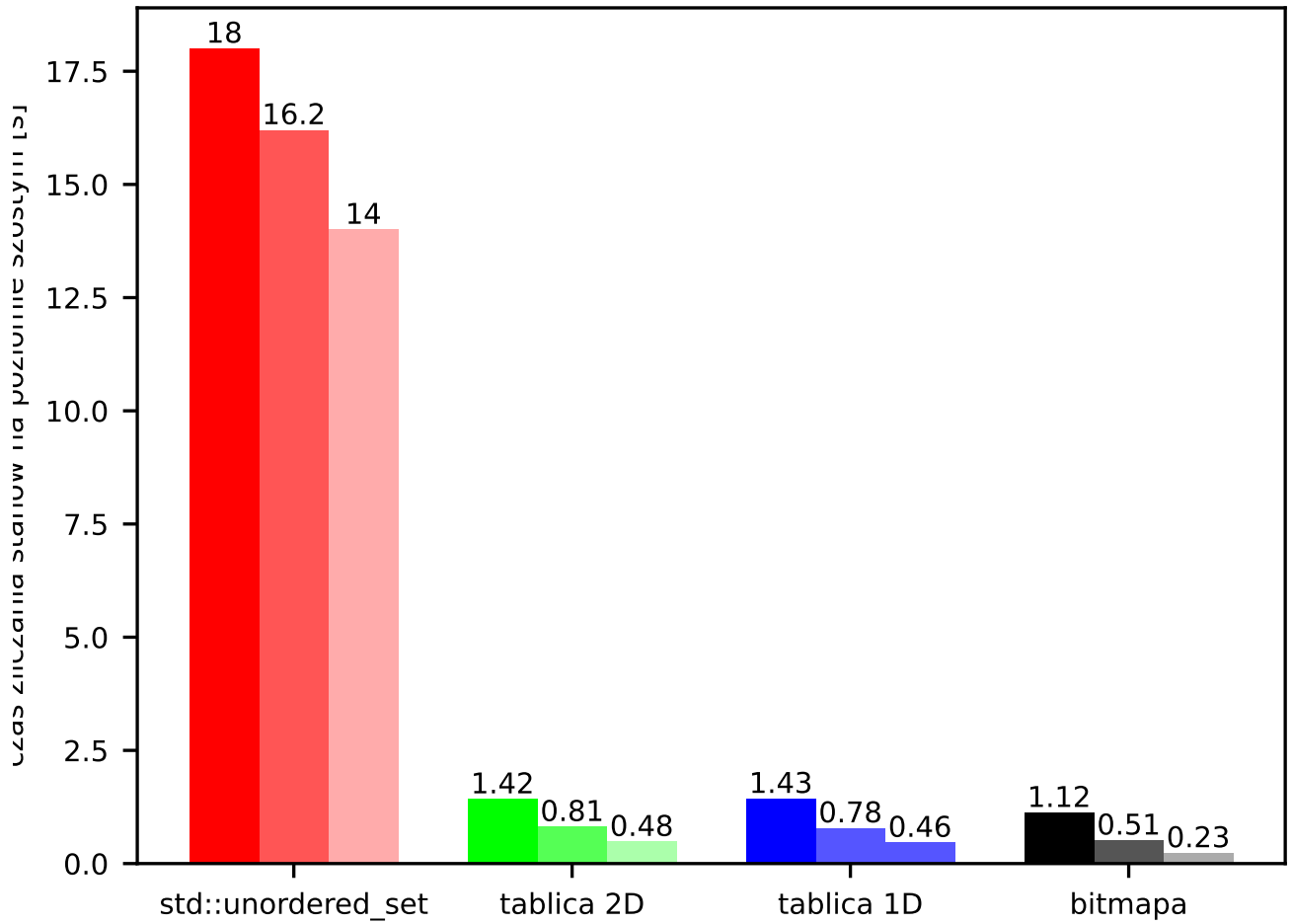
Liczba stanów na poszczególnych poziomach drzewa, zaczynając od podanej planszy (ruch białych, graczom zostały 4 pionki):

głębokość	liczba stanów
1	17
2	242
3	4094
4	58228
5	929238
6	12747908



Przykład II

porównanie wydajności poszczególnych sposobów reprezentacji



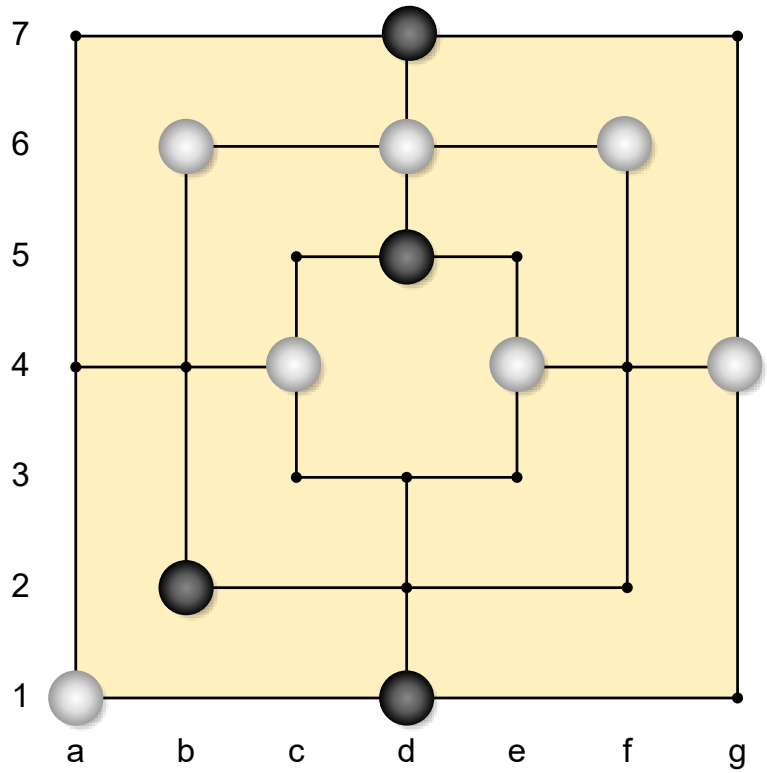
pierwszy słupek: `set_move(std::format(...))`; drugi słupek: `m_move += ...`; trzeci słupek: współrzędne w bajtach

Przykład III

drugi etap gry

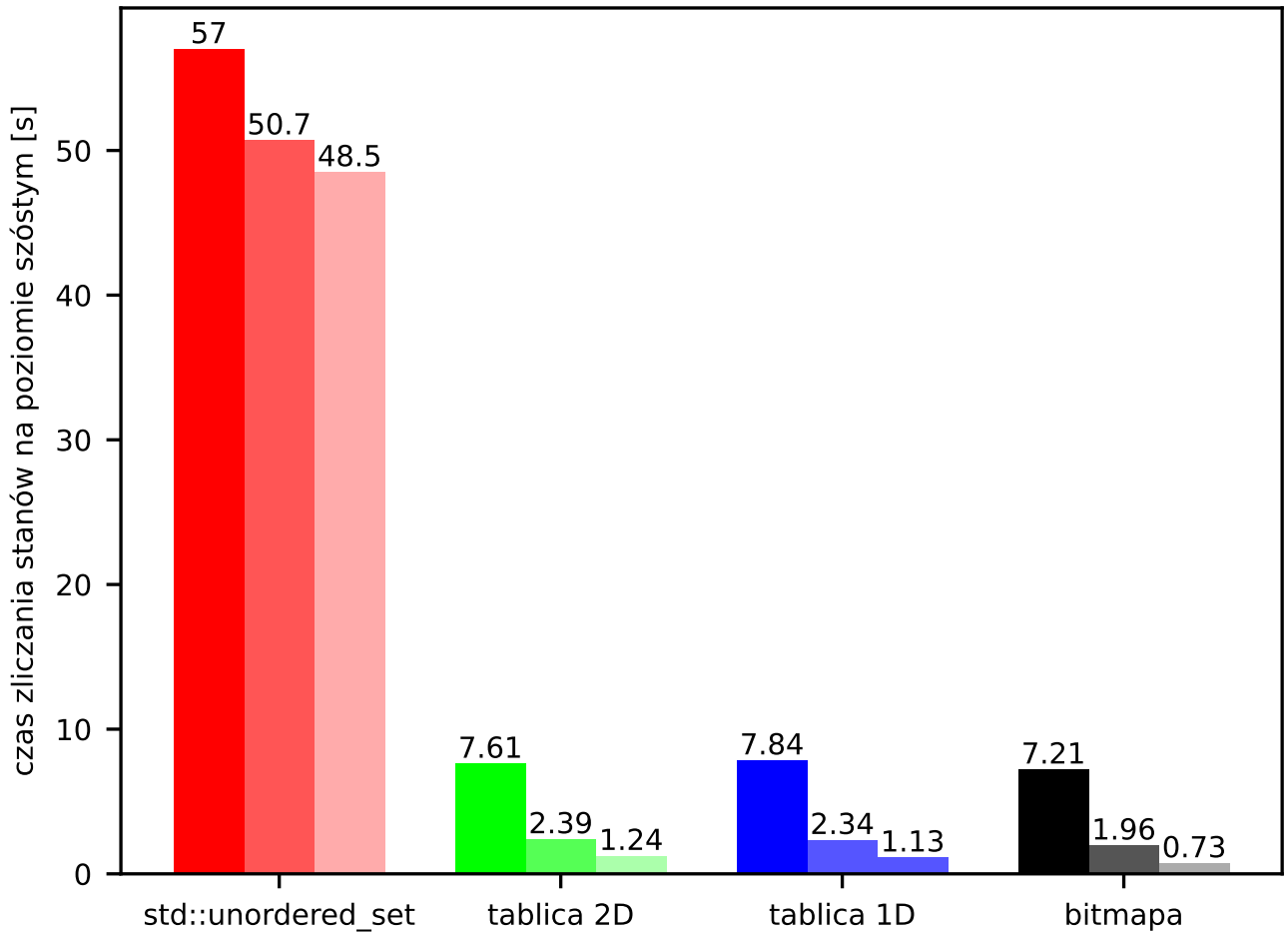
Liczba stanów na poszczególnych poziomach drzewa, zaczynając od podanej planszy (ruch białych):

głębokość	liczba stanów
1	15
2	251
3	3509
4	100781
5	1507836
6	49700133



Przykład III

porównanie wydajności poszczególnych sposobów reprezentacji



pierwszy słupek: `set_move(std::format(...))`; drugi słupek: `m_move += ...`; trzeci słupek: współrzędne w bajtach

Ogólne uwagi

- Klasa reprezentująca stan gry ma dziedziczyć po klasie szablonowej `game_state` — w parametrach szablonu wskazać nazwę klasy i typ reprezentujący posunięcie.
- Ruch gracza koduje flaga `stm` typu `side_to_move` — proszę pamiętać o przełączaniu jej w stanach potomnych.
- W stanach potomnych proszę pamiętać o ustawieniu nazwy ruchu (pole `m_move` lub metoda `set_move()`), by później łatwiej znajdować szukany stan na podstawie ruchu.
- Przygotuj metodę oceniającą pozycję — w wersji podstawowej niech zwraca różnicę liczby pionków białych i czarnych.

Szkic klasy

```
class ... : public game_state<..., std::string>
{
    // plansza jako tablica jedno/dwuwymiarowa
    // albo dwa obiekty typu std::unordered_set

    // liczba pionków do rozmieszczenia:
    // != 0 - pierwszy etap
    // == 0 - drugi/trzeci etap
    // na początku = 18 i z każdym ruchem dekrementacja

    // (być może) liczba pionków każdego z graczy
    // metody: generujące potomków w pierwszej i drugiej fazie,
    //         testująca obecność młynka, rozwiązująca młynek
public:
    // konstruktor ustawi pole "stm" (czyj ruch) na "max_player"
    ...() : game_state(side_to_mode::max_player) {}
    // pozostałe metody
}
```

Sugerowana kolejność realizacji zadania

1. przygotuj klasę reprezentującą pozycję na planszy,
2. napisz metodę, do której przekażesz współrzędne badanego pola, a ona w wyniku zwróci informację, czy pionek na wskazanym polu wchodzi w skład młynka (gdy pionek znajduje się na parzystym polu, to sprawdź dwa kolejne i dwa poprzednie pola, czy znajdują się tam pionki tego samego koloru, natomiast gdy pionek znajduje się na nieparzystym polu, to sprawdź sąsiednie pola oraz pola między kwadratami), np. `bool czy_mlynek(... i, ... j) const` (gdzie `i` to nr kwadratu, a `j` to pole w kwadracie w przypadku reprezentacji za pomocą tablicy dwuwymiarowej; jeśli użyjesz typu bez znaku, to będzie można bezpiecznie policzyć *modulo* dla ujemnej wartości),

Sugerowana kolejność realizacji zadania cd.

3. napisz metodę, która wygeneruje stany potomne w przypadku, gdy któryś z graczy zbudował młynek, a decyzję o tym, komu zabierać pionki, ustalisz na podstawie wartości pola `stm`, np. `std::vector<...> rozwiąż_młynek() const` lub `void rozwiąż_młynek(std::vector<...>&) const`,
4. przygotuj metodę, która wygeneruje stany potomne w pierwszej fazie gry i zweryfikuj jej poprawność,
5. przygotuj metodę, która wygeneruje stany potomne w drugiej fazie gry i zweryfikuj jej poprawność,
6. przygotuj metodę umożliwiającą rozgrywkę (człowiek kontra maszyna, maszyna kontra człowiek).

Szkic metody rozwiązującej młynek

Można też zebrać położenie pionków, które nie tworzą młynka, żeby uniknąć dwukrotnego wywołania metody `czy_młynek`.

```
void rozwiąż_młynek(std::vector<...>& ...) const
{
    sprawdź, czy wszystkie pionki przeciwnika są w młynku

    dla każdego pionka przeciwnika:
        jeżeli (wszystkie w młynku) lub !czy_młynek():
            utwórz kopię stanu
            zabierz z kopii tego pionka
            w kopii do ruchu (pole m_move) dopisz
              położenie zabranego pionka
            dodaj stan do tablicy wynikowej
}
```

Szkic generowania stanów w pierwszej fazie gry

```
std::vector<...> ...() const
{
    dla każdego wolnego pola:
        utwórz kopię stanu
        umieść w kopii pionka gracza na tym polu
        w kopii zmniejsz liczbę pionków do wyłożenia
        ustaw w kopii ruch jako indeks pola
        jeżeli powstał młynek w wyniku tego ruchu:
            rozwiąż młynek
        w przeciwnym razie:
            dodaj stan do tablicy wynikowej

    zmień "stm" na przeciwny w tablicy wynikowej stanów
}
```

Szkic generowania stanów w drugiej fazie gry

Druga faza jest podobna do pierwszej, z tą różnicą, że nie kładziemy pionków, a przesuwamy je na sąsiednie wolne pola. Jeżeli gracz, któremu przypada ruch, został z trzema pionkami, to może je przesuwać na dowolne puste pole.

Zliczanie stanów

Funkcję trzeba poprawić, aby zliczała stany na poszczególnych poziomach — w przeciwnym razie trzeba ją wywołać dla każdej wartości `max_lvl` od 1 do 6.

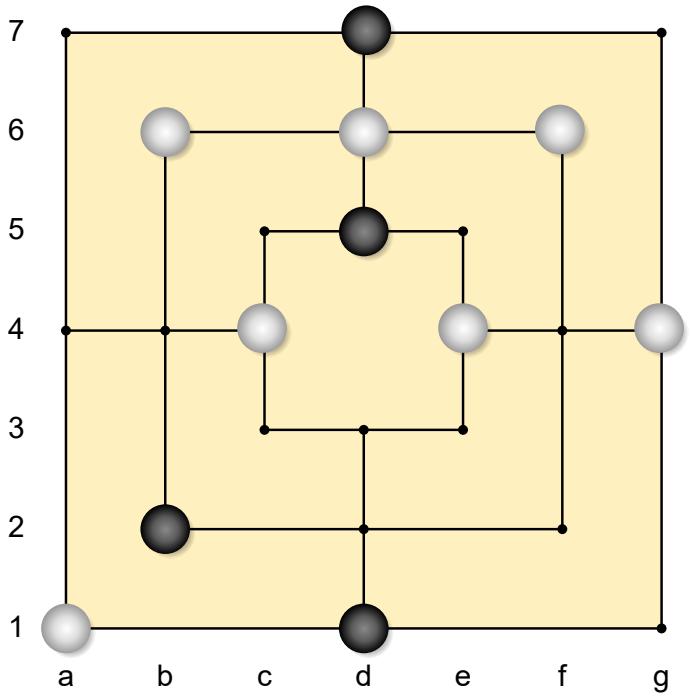
```
template<typename T>
uint64_t dfs(uint64_t max_lvl,
             const T &s,
             std::vector<T> (T::*children)() const)
{
    if (max_lvl == 0) return 1;

    uint64_t cnts = 0;
    for (const auto &t : std::invoke(children, s))
        cnts += dfs(max_lvl - 1, t, children);

    return cnts;
}
```

Przykład

Białe zaczynają i wygrywają (głębokość przeszukiwanie $D = 6$).



Przykładowa zwycięska sekwencja ruchów dla gracza białego (wydruk `get_pv()` po zakończeniu przeszukiwania):

0	7		*1*	21	20
12	11	1	*2*	20	22
11	12	5	*3*		

Drugim zwycięskim ruchem jest ruch pionka z `b6` na `b4`.